DeployGuard

Remaining Gaps - Verified Worklist

Post Attempt 13 | 13 August 2026

Purpose: A clean action list containing only unresolved or not-yet-certified lifecycle risks found in the inspected backend/workflow snapshot, reconciled against the later Codex execution results through successful Destroy Attempt 13.

Evidence rule: Direct source inspection is treated as code evidence. Later Codex results are treated as execution evidence. A later result closes an item only when it explicitly proves or reports that exact issue was fixed.

Recommended use: fix P0 items first, then P1 hardening, then run the final lifecycle certification sequence.

1. Executive status

The core DeployGuard path is no longer in the earlier dispatch/Destroy failure state. The later live results close the previously observed GitHub dispatch and Destroy blockers. The remaining work is concentrated in Redeploy, Rollback, state projection, generation failure cleanup, retired-generation cleanup, and final certification.

Area	Status	What is proven / missing	Basis
First Deploy to LIVE	CLOSED	Attempt 4 reached LIVE after promotion and health verification.	Execution evidence in conversation
GitHub dispatch + immutable run ID	CLOSED	Attempt 9 returned HTTP 200 and exact GitHub run ID 31674555833.	Later Codex live result
Destroy Terraform contract	CLOSED	Attempt 10 passed the prior platformFoundation/routing parser failure.	Later Codex live result
Full Destroy + project extinction	CLOSED	Attempt 13 succeeded; project API 404, project/generation/pipeline rows gone, ECR/runtime secret/state absent, caller workflow removed.	Later Codex live result
Redeploy	OPEN	No end-to-end live proof after the generation architecture changes; source shows a concrete candidate-route collision risk.	Source inspection
Rollback	OPEN	No end-to-end live proof; source shows immutable evidence fields are not persisted on rollback operation creation.	Source inspection

Work order: P0-1 Redeploy routing -> P0-2 Rollback evidence -> P1 state projection -> P1 generation failure cleanup -> P1 retired cleanup -> P1 Destroy certification coverage -> P2 reproducibility -> final lifecycle certification.

2. P0-1 - Redeploy candidate routing can collide

Status: OPEN - source-proven risk. No later Codex result reports changing candidate-priority allocation or successful-promotion candidate-rule cleanup.

Exact source evidence

- The project/environment route allocates one stable listener priority and reuses the same ProjectEnvironmentRoute for that project/environment. [S1: backend/src/projects/deployment-generation.service.ts:L214-L231]

- Every new deploy/rollback generation derives verificationPriority as listenerPriority + 20,000. [S1: backend/src/projects/github-actions-deployment.service.ts:L348-L353; L1231-L1235]
- The reusable workflow creates the candidate ALB listener rule at runtime_config.routing.verificationPriority. [S2: deployguard-reusable.yml:L716-L727]
- The inspected workflow has no successful-promotion candidate-rule deletion; rule deletion appears in Destroy/compensation paths. Retired-generation cleanup is hard-disabled with if: false. [S2: L1067-L1077; L1250-L1273]

Failure mode G001 can leave its candidate listener rule behind after becoming LIVE. G002 then receives the same verification priority. A new ALB candidate rule can therefore collide with residue from G001, blocking Redeploy even though generation identities and Terraform state keys are isolated.

Required fix Persist a generation-specific candidate listener priority (or an equivalent collision-free candidate routing identity) instead of reusing project listenerPriority + 20,000. Keep the production listener priority stable per project/environment. Successful promotion must also make the candidate route safely removable without affecting the production route.

Done when

- G001 is LIVE and remains physically present.
- A Redeploy creates G002 without deleting/adopting G001 and without ALB priority collision.
- G002 passes candidate health, promotion, production HTTP verification and becomes LIVE.
- G001 becomes RETIRED/CLEANUP_PENDING without blocking a later G003 Redeploy.
- Production host resolves to the new LIVE generation only.

3. P0-2 - Rollback operation does not persist required immutable evidence

Status: OPEN - source-proven defect. No later result reports a live Rollback or a fix to rollback operation evidence persistence.

- Rollback clones the prior runtime configuration and creates a new candidate generation. [S1: github-actions-deployment.service.ts:L327-L389]
- The new rollback ProjectPipelineRun is persisted without configurationSnapshotId or databaseServiceBindingId. [S1: L390-L424]
- Promotion later requires candidate.configurationSnapshotId to equal operation.configurationSnapshotId and candidate.databaseBindingId to equal operation.databaseServiceBindingId. [S1: L742-L760]

Failure mode The rollback candidate can return valid source snapshot/binding evidence, but the new rollback operation has null identity columns. beginPromotion can reject the candidate as not matching the immutable operation before cutover.

Required fix When creating the rollback operation, persist the authoritative configurationSnapshotId and databaseServiceBindingId that the rollback runtime actually carries. Keep these values consistent across operation columns, immutable dispatch inputs, candidate evidence and promotion validation. Rollback must remain application-only and must not mutate project persistence.

Done when

- Deploy release A -> Redeploy release B -> both have immutable evidence.

- Rollback to A creates a new immutable operation/candidate, reuses A known-good image evidence and passes beginPromotion.
- Production route returns to the rollback target; exact image/task-definition evidence is verified.
- Database/EFS/project persistence identity remains unchanged.

4. P1-1 - Current-state projection mixes runtime state with latest operation state

Status: OPEN - source-proven semantic defect. The later dispatch fix reported that the current-state matrix passed, but it did not report changing project-current-state.service.ts or closing these projection rules.

- Stable release lookup is constrained to latest.generationId instead of the authoritative live generation. [S1: current-state/project-current-state.service.ts:L201-L212]
- latestAttempt.releaseRevision is the operation attempt number, so non-release operations can appear as release numbers. [S1: L214-L243]
- Any failed latest GitHub operation maps the project to failed_application, even when a prior LIVE generation remains healthy. [S1: L317-L337]
- AWS LIVE health evidence is queried using latestAttempt.generationId rather than route.liveGenerationId. [S1: L107-L115]

Required behavior Separate the authoritative runtime/release state from the latest operation state. Stable release and health must be derived from the stable route / liveGenerationId. Latest operation must be shown independently. A failed Destroy or failed Redeploy must not downgrade a still-serving LIVE release to FAILED.

Done when - required state matrix

- LIVE G001 + failed Destroy dispatch -> Runtime LIVE/Healthy; Latest operation = Destroy Failed.
- LIVE G001 + G002 DEPLOYING -> stable release/health still point to G001 while latest operation shows G002 progress.
- LIVE G001 + G002 FAILED -> G001 remains LIVE/Healthy; G002 failure is shown separately.
- Destroy success -> project is gone/404, not LIVE.
- Destroy/Retry attempts never become display releases such as "Release 7".

5. P1-2 - Failed candidate preparation can strand a generation in DEPLOYING

Status: VERIFY CURRENT SOURCE, then fix if unchanged. Later dispatch/Destroy work touched github-actions-deployment.service.ts, but no result states this Deploy/Reset-Fresh failure path was corrected.

- deploy() creates a candidate, dispatches, then on error calls markFailed with generation.createdByOperationId || generation.id from the pre-dispatch in-memory object. [S1: github-actions-deployment.service.ts:L173-L188]
- markFailed refuses the update when activeOperationId/createdByOperationId does not equal the supplied operation ID. [S1: deployment-generation.service.ts:L156-L172]
- resetAndDeployFresh() creates a candidate and dispatches without an equivalent terminal cleanup catch. [S1: github-actions-deployment.service.ts:L250-L262]

Required fix Terminal pre-dispatch/dispatch failure must always transition the exact candidate generation to FAILED and clear route.candidateGenerationId using the persisted operation ID. Do not use a stale pre-dispatch generation object as operation authority.

Done when

- Inject failure after candidate creation but before GitHub run creation: operation becomes FAILED, generation becomes FAILED, candidateGenerationId clears.
- Repeat for RESET_FRESH and Retry paths.
- A later Deploy can create a new candidate without manual database repair.

6. P1-3 - Retired-generation cleanup is disabled

Status: OPEN in inspected workflow. The last inspected reusable workflow explicitly disables retired cleanup. Later Destroy fixes do not report enabling an independent post-promotion cleanup mechanism.

- The workflow contains "Clean retired generation independently" but gates it with if: false. [S2: deployguard-reusable.yml:L1263-L1273]
- Promotion marks the previous generation RETIRED and cleanupRequired=true. [S1: deployment-generation.service.ts:L130-L146]

Required fix Implement cleanup as an operation that starts only after the backend has committed the new generation LIVE. Cleanup failure must be non-blocking: RETIRED -> CLEANUP_PENDING, while new deployments remain allowed. It must delete only exact retired-generation resources/state and never shared/project-persistence resources.

Done when

- After G002 becomes LIVE, G001 cleanup runs independently.
- A forced cleanup failure leaves G001 CLEANUP_PENDING and does not affect G002 or block G003.
- A later cleanup retry converges G001 to CLEANED using exact generation ownership.

7. P1-4 - Expand final Destroy certification evidence

Status: CERTIFICATION GAP, not a current Destroy failure. Attempt 13 proves successful extinction for the tested project. The supplied result explicitly verifies several key resources, but it does not enumerate every project/generation-owned AWS resource class.

Attempt 13 is sufficient to close the previously failing Destroy path for this test case. Before calling the platform production-certified, capture exact absence evidence for every resource class DeployGuard can create, especially in a persistence-enabled project.

- Generation resources: ECS service, target group, candidate listener rule, security group, CloudWatch log groups, execution/task IAM roles/policies, task-definition revisions.
- Project resources: ECR, runtime secret, stable listener rule, project Terraform state.
- Managed database/persistence path when enabled: EFS filesystem/access point/mount targets, DB ECS service, Cloud Map namespace/service, DB secrets, DB task-definition revisions and project-persistence state.
- Shared foundation must remain: shared VPC/subnets, shared ECS cluster, shared ALB/listener.

Done when One Destroy of a persistence-enabled project produces an evidence bundle listing exact resource identities and proving absence/deleted state for all owned resource classes while proving the shared foundation still exists.

8. P2 - Workflow reproducibility hardening

Status: HARDENING, not a functional blocker. The inspected immutable reusable workflow still calls third-party actions by major-version tags and Terraform providers by version ranges.

- actions/checkout@v4 and aws-actions/configure-aws-credentials@v4 are major-version refs. [S2: L71-L81]
- hashicorp/setup-terraform@v3 is a major-version ref. [S2: L336-L341]
- Terraform provider constraints are aws ~> 5.0 and random ~> 3.0 in generated modules. [S2: L348-L358; L613-L623]
- actions/upload-artifact@v4 is also a major-version ref. [S2: L1403-L1413]

Required hardening Pin external GitHub Actions to exact commit SHAs and use an exact Terraform provider lock/version strategy for the production workflow. Keep Dependabot/Renovate or a deliberate update process for controlled upgrades.

9. Final certification sequence after P0/P1 fixes

Run this as one controlled lifecycle. Do not treat unit/script PASS as a substitute for the live checkpoints below.

1. Fresh project after extinction: create a new project identity for the same repository and deploy to LIVE. Confirm no old project/generation identity is adopted.
2. Redeploy: change the application commit and deploy a new generation while the previous LIVE generation still physically exists. Confirm collision-free candidate routing and LIVE promotion.
3. Controlled failed Redeploy: intentionally fail the new candidate before promotion. Confirm previous LIVE stays healthy and authoritative.
4. Retry: retry the failed candidate. Confirm a new immutable operation reuses the same failed candidate generation and exact commit/action, then succeeds.
5. Rollback: roll back to the previous known-good immutable release. Confirm exact image/task-definition evidence, route cutover and unchanged persistence.
6. Retired cleanup: verify previous retired generation converges to CLEANED or remains CLEANUP_PENDING without blocking the next deployment.
7. Destroy: destroy the final project and capture the exhaustive owned-resource absence evidence plus shared-foundation survival.
8. UI/current-state review during each transition: Overview and Pipeline must show the same authoritative runtime state and latest operation semantics.

10. Closed items - do not reopen without new evidence

- Destroy candidate-generation binding / generation_not_deploying before dispatch - fixed by removing Deploy-only binding from Destroy.
- GitHub dispatch/run discovery for the tested path - live-proven by Attempt 9 exact run ID.
- Destroy Terraform runtime contract platformFoundation/routing parser failure - fixed and passed former failure point in Attempt 10.

- Destroy undefined PROJECT_PREFIX - fixed before successful Attempt 13.
- RuleNotFound handling for already-deleted ALB rules - fixed before successful Attempt 13.
- Immediate stale GitHub Contents read during managed caller-workflow removal - bounded absence verification added and Attempt 13 succeeded.
- Core Destroy extinction for the tested project - Attempt 13 succeeded and project API returned 404.

11. Source register and confidence

S1 - Direct code source: uploaded backend ZIP 4af9eb7b-9126-4940-8137-282afcc0c27b.zip. 757 files were extracted and inspected. Line references in this report refer to that snapshot.

S2 - Direct workflow source: uploaded deployguard-reusable.yml text (Pasted text(20260813-063729).txt), 1,414 lines. Line references in this report refer to that uploaded workflow snapshot.

S3 - Direct caller source: react-pomodoro deployguard.yml pasted in the conversation, pinned at that time to ee6d8a48634429a9a3594825801c369670dc68bf.

E1-E4 - Execution evidence: later Codex results pasted in the conversation for Attempt 4, Attempt 9, Attempt 10 and Attempt 13. These are used only to close the exact issues they explicitly report as fixed/live-proven.

Important currency rule: later fixes changed github-actions-deployment.service.ts and deployguard-reusable.yml after the inspected snapshot. Before editing any OPEN source-proven item, first confirm the cited pattern still exists in the current branch. If it no longer exists, do not reimplement it; run the corresponding Done-When live test instead.

12. One-page worklist

Priority	Work item	Exit condition
P0	Redeploy candidate routing	Persist generation-specific candidate priority; prove G001 -> G002 -> G003 without collision.
P0	Rollback immutable evidence	Persist rollback configurationSnapshotId/databaseServiceBindingId; live-prove rollback.
P1	Current-state authority	Separate LIVE runtime/release from latest operation; fix release numbering and health generation.
P1	Candidate terminal transitions	Guarantee failed preparation/dispatch marks exact candidate FAILED and clears route candidate.
P1	Retired cleanup	Enable post-LIVE independent cleanup with CLEANUP_PENDING retry semantics.
P1	Destroy certification coverage	Run persistence-enabled Destroy and prove exact absence of all owned resource classes.
P2	Dependency pinning	Pin external Actions/provider resolution for reproducible immutable workflow execution.
CERT	Lifecycle proof	Fresh -> Redeploy -> fail -> Retry -> Rollback -> cleanup -> Destroy, with synchronized UI evidence.

Recommended next action: Start with P0-1 Redeploy candidate routing. Do not modify the now-live-proven Destroy path unless the Redeploy/cleanup fix requires a narrowly shared routing primitive.